

CFGS Desenvolupament d'Aplicacions Multiplataforma

CFGS Desenvolupament d'Aplicacions Web

Bases de dades

Jordi Quesada
jordi.quesada@iticbcn.cat

Llicència.



Atribución-NoComercial-CompartirIgual
4.0 Internacional (CC BY-NC-SA 4.0)

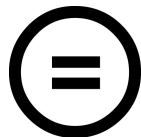
El contingut està disponible sota la llicència
[Creative Commons Reconeixement-NoComercial-CompartirIgual](https://creativecommons.org/licenses/by-nc-sa/4.0/)



Reconeixement — Heu de reconèixer l'autoria de manera apropiada, proporcionar un enllaç a la llicència i indicar si heu fet algun canvi. Podeu fer-ho de qualsevol manera raonable, però no d'una manera que suggereixi que el llicenciador us dóna suport o patrocina l'ús que en feu.



NoComercial — No podeu utilitzar el material per a finalitats comercials.



CompartirIgual — Si remescleu, transformeu o creeu a partir del material, heu de difondre les vostres creacions amb la mateixa llicència que l'obra original.

EXTENSIÓ PROCEDIMENTAL

Introducció

Introducció

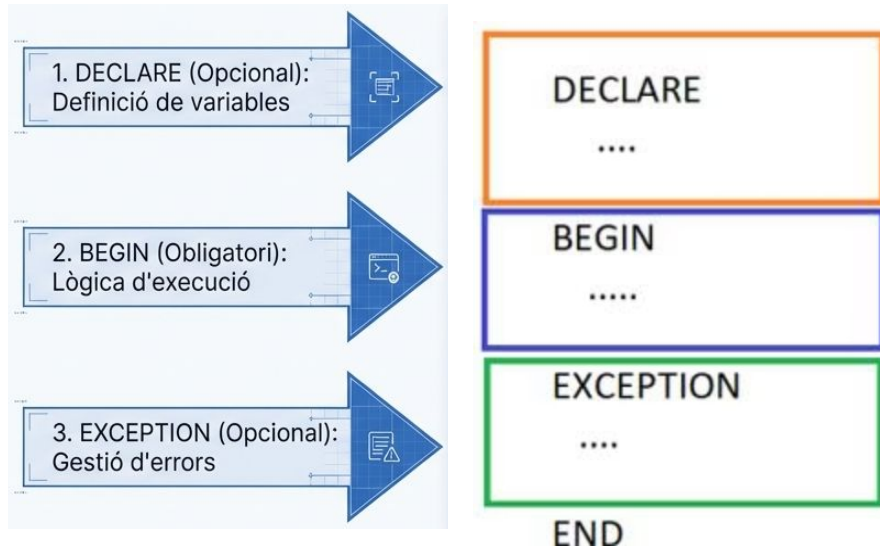
- La majoria dels SGBD incorporen mecanismes per estendre la seva funcionalitat mitjançant llenguatges procedimentals.
- PL/SQL (Procedural Language/SQL) expandeix les capacitats d'Oracle permetent executar lògica de programació (if, for, while...) directament al motor de la base de dades

Oracle
PL/SQL

PostgreSQL
pgPL/SQL

Introducció

- La estructura mínima de treball a PL/SQL s'anomena **bloc**
- Cada bloc té 3 parts:



Introducció



Sintaxi bàsica

- Case insensitive.
- Cada instrucció acaba amb ;
- Sortida per pantalla:

`DBMS_OUTPUT.PUT_LINE(text).`

- Al terminal cal activar la sortida per pantalla amb `set serveroutput on.`

Introducció



Primer codi: Hola món!

```
1
2 BEGIN
3     DBMS_OUTPUT.PUT_LINE('Hola k ase');
4 END;
5 /
```


Introducció



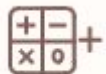
Variables

- Declarades a la zona DECLARE:

nom tipus [:= valor]

- Assignació amb :=
- Comparació amb =

Introducció



Operadors

- Aritmètics:

+ - * /

- Concatenació de strings:

||

Conditionals

Conditionals

Els conditionals permeten executar codi diferent segons el resultat d'una condició.

En PL/SQL, els tres formats més habituals són IF, ELSIF i CASE.

Conditionals

IF / END IF

Serveix per a condicions simples o amb ELSE.

És l'opció més directa quan només hi ha dues alternatives.

```
1 DECLARE
2     nota integer:=6;
3 BEGIN
4     IF nota >= 5 THEN
5         DBMS_OUTPUT.PUT_LINE('Aprovat');
6     ELSE
7         DBMS_OUTPUT.PUT_LINE('Suspès');
8     END IF;
9 END;
10 /
```

Conditionals

ELSIF

S'utilitza quan hi ha múltiples condicions encadenades.

Permet comprovar diversos rangs o casos dins d'un mateix bloc.

```
1 DECLARE
2     nota integer:=6;
3 BEGIN
4     IF nota >= 9 THEN
5         DBMS_OUTPUT.PUT_LINE('Excel·lent');
6     ELSIF nota >= 7 THEN
7         DBMS_OUTPUT.PUT_LINE('Bé');
8     ELSIF nota >= 5 THEN
9         DBMS_OUTPUT.PUT_LINE('Aprovat');
10    ELSE
11        DBMS_OUTPUT.PUT_LINE('Suspès');
12    END IF;
13 END;
14 /
```

Conditionals

CASE

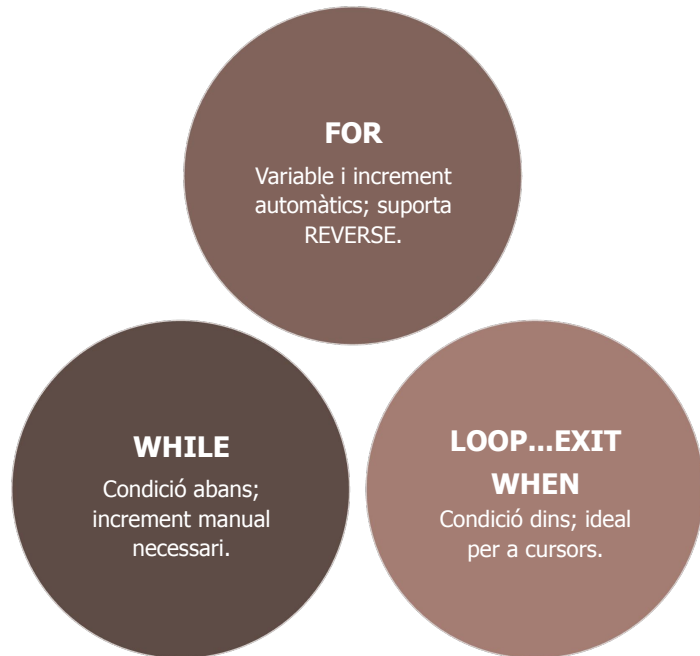
És més llegible quan hi ha molts valors possibles, especialment si es compara una mateixa variable amb opcions concretes.

```
1 DECLARE
2     dia integer:=6;
3 BEGIN
4     CASE dia
5         WHEN 1 THEN DBMS_OUTPUT.PUT_LINE('Dilluns');
6         WHEN 2 THEN DBMS_OUTPUT.PUT_LINE('Dimarts');
7         WHEN 3 THEN DBMS_OUTPUT.PUT_LINE('Dimecres');
8         WHEN 4 THEN DBMS_OUTPUT.PUT_LINE('Dijous');
9         WHEN 5 THEN DBMS_OUTPUT.PUT_LINE('Divendres');
10        ELSE DBMS_OUTPUT.PUT_LINE('Altres dia');
11    END CASE;
12 END;
13 /
```

Bucles

Bucles

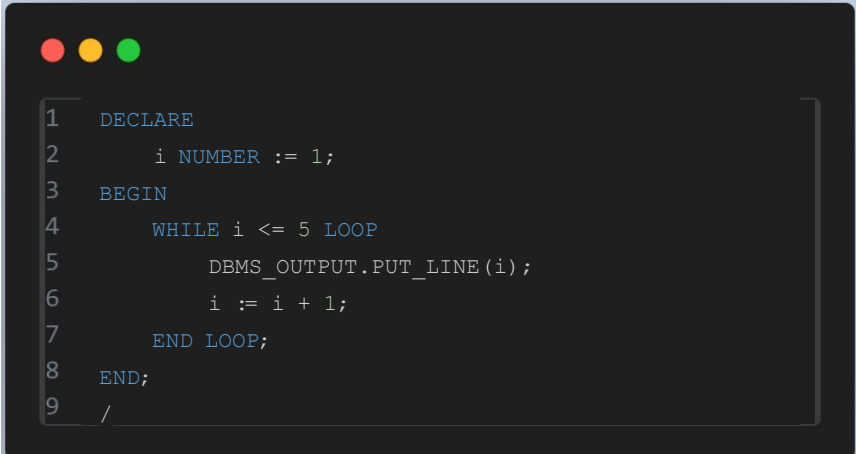
- Hi ha 3 formes de fer bucles a PL/SQL:
 - WHILE
 - FOR
 - LOOP...EXIT WHEN.
- Són alternatives equivalents per repetir instruccions segons la lògica que necessitem.



Bucles

While

La condició es comprova abans de cada iteració, i cal incrementar la variable manualment.



```
1 DECLARE
2     i NUMBER := 1;
3 BEGIN
4     WHILE i <= 5 LOOP
5         DBMS_OUTPUT.PUT_LINE(i);
6         i := i + 1;
7     END LOOP;
8 END;
9 /
```

Bucles

For

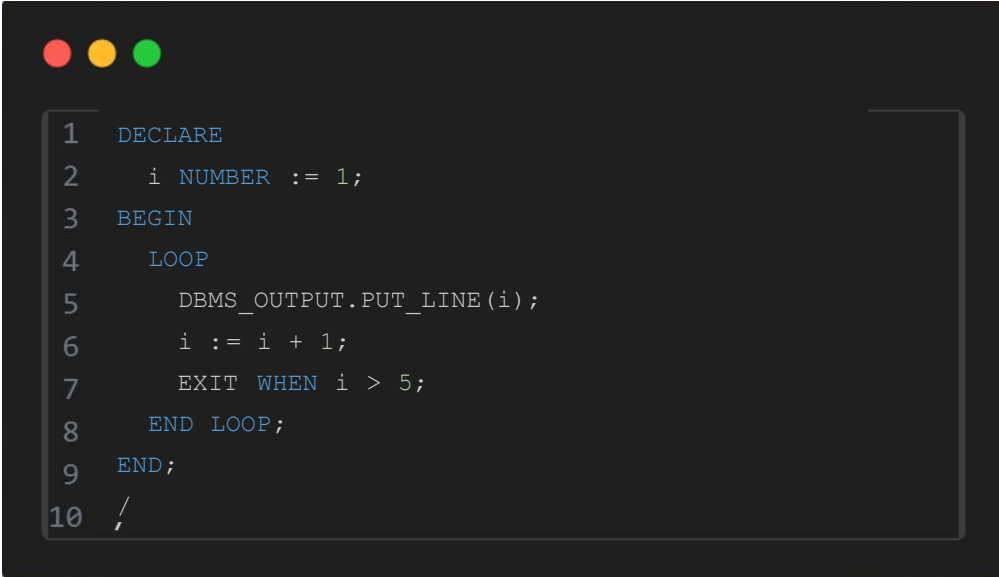
La variable i l'increment són automàtics, i a més suporta REVERSE.

```
1 BEGIN
2   FOR i IN 1..5 LOOP
3     DBMS_OUTPUT.PUT_LINE(i);
4   END LOOP;
5 END;
6 /
```

Bucles

Loop .. Exit when

La condició es comprova dins del bucle, i és especialment útil quan treballem amb cursors.



```
1  DECLARE
2      i NUMBER := 1;
3  BEGIN
4      LOOP
5          DBMS_OUTPUT.PUT_LINE(i);
6          i := i + 1;
7          EXIT WHEN i > 5;
8      END LOOP;
9  END;
10 /
```

Procediments

Procediments

- Podem encapsular els blocs pl/sql
 - Els podem reutilitzar
 - Podem parametritzar-los

- Es defineixen:

```
CREATE [OR REPLACE] PROCEDURE nom IS  
Bloc PL/SQL
```

- S'executen amb:

```
execute nom
```

Procediments

- Podem indicar paràmetres d'entrada al procediment
- Als paràmetres s'indica el tipus però no la precissió

Number

varchar2

char

Number(8,2)

varchar2(100)

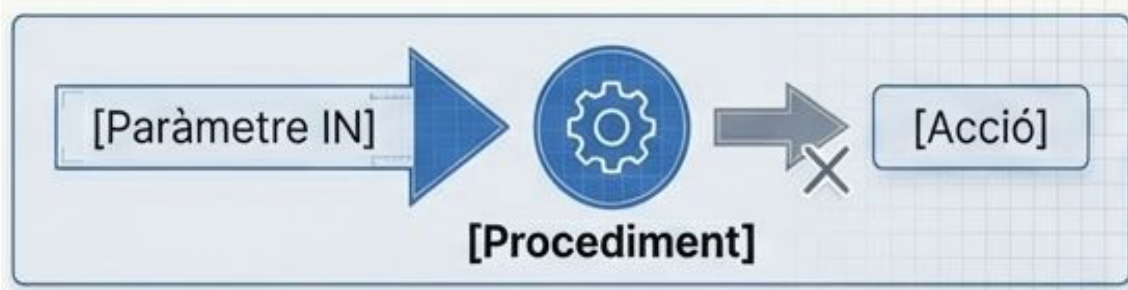
char(20)

- Definició:

```
CREATE [OR REPLACE] PROCEDURE nom (param1 tipus1, param2 tipus2...) IS  
Bloc PL/SQL
```

Procediments

- Els procediments simplement fan accions o mostren resultats per pantalla però no retornen cap valor



Funcions

Funcions

- Són similars als procediments, però:
 - No haurien de mostrar res per pantalla
 - Retornen un valor



Funcions

- Es defineixen:

CREATE OR REPLACE FUNCTION nom (params) RETURN tipus IS

Bloc PL/SQL

- Per executar-les:
 - Des d'altres procediments o funcions
 - Directament des de SQL
 - NO es pot fer execute!!